

Operating Systems Design & Implementation

Prerequisites:

CS 361 [Computer Systems](#) or equivalent courses

Proficient C programming skills in a shell environment

Some familiarity **with (x86) assembly**

Syllabus

General Course Information

- CS 461. Operating Systems Design and Implementation. 3 or 4 hours. Elective.
- Prerequisites: CS 361.
- Synchronous in-person & recorded
- MWF 12-12:50PM in TBH 180G
- Instructor: Dr. Jakob Eriksson
- Office Hours: Wed 2 pm, Fri 10 am (in SEO 1309: elevator to 12, then stairs)
- TA: Wenshao Zhong
- TA Office Hours: Tuesday 2-3PM and Thursday 11-12PM on [Zoom](#)
- Course Discussion: [Piazza](#)

Course Objectives

By the end of this course, you will have a good understanding of the main elements that work together to form an Operating System (OS). You will have significant experience with C programming for low-level OS development, and some additional exposure to low-level application development. You will have acquired familiarity with standard tools for revision control, debugging, and system diagnostics.

While the class combines theory and practice, there will be a focus on practical demonstrations in lecture and extensive hands-on programming experience. Plan to spend 10 +/- 5 hours per week on homework programming, depending on your current proficiency.

Materials

You will be provided a small booklet containing the source code of the xv6 operating system.

All other materials are freely available online.

- The 64-bit xv6 source code: <https://github.com/bitmlab/xv6-public>
- The original xv6 [book](#) and [source code](#) (32-bit x86).
- [Operating Systems: Three Easy Pieces](#)

Evaluation

The emphasis of this class is on learning by doing. That said, evaluation this semester is primarily through quizzes and exams, not graded assignment turn-ins. Quizzes and exams will be designed with the intent that if you did the assignment, and have a good understanding of what you did in the assignments and why, then you will do well on both quizzes and exams.

Evaluation will be done through:

- 7 programming assignments with associated quizzes (total **49%**)
 - for each assignment, **1%** for a turn-in that meets requirements and passes any automated tests provided
 - for each assignment, **6%** based on an in-class quiz, conducted on the day of the due date.
- one midterm exam (**20%**, in-person on **Wed, Oct 11 during regular lecture**)
- one comprehensive final exam (**30%, in-person during finals week**)
- class participation, defined as improving the class experience for others (**1%**)

Exams and quizzes will be pen and paper based. You are allowed to bring any paper materials you want to the exam, but you **must bring the xv6 booklet**. All annotations in the booklet, and elsewhere are permissible, just no electronics, and no peeking!

The percentages above reflect the value of each individual element. However, every element except the final exam will be counted only if it improves your total course score. Procedurally, we calculate the course score starting with the final exam percentage. Then, in best-to-worst order, we incrementally include each of the 16 elements above in the course score, as long as the score improves.

Thus, there will be many opportunities to make up for lost ground over the course of the semester. Some examples follow:

- if you ace (100%) the final exam, you automatically aced the course overall.
- if you bombed (0%) the midterm, it won't be counted.
- if you bomb an assignment, it won't be counted.
- **if you bomb the final, it will be counted.**
- if you ace all the assignments, but bomb both exams, your course score will be 49%+participation.
- if you ace assignments and midterm, but bomb the final exam, your course score will be 69%+participation.

Note Well: exams in my classes are hard, and tend to have mean and median around 50%. Quizzes will be easier. Thus, doing each assignment and taking the associated quiz is the easiest path to a high course score. It is also the best way to learn the course material.

Grades will be based on a relative curve, determined after the final exam. Graduate and undergraduate students will be curved separately. A rough estimate, not to taken as the actual curve for this class, and based on results from past years, which used a different scheme to calculate course totals: *A: 70%, B: 50%, C: 35%, D: 20%*.

Course Participation Credit

A 1% credit is available for course participation. To claim your credit, email the instructor on the last day of class with a one-paragraph statement explaining how you made the class better for others. It could be examples of errors you pointed out during lecture, insightful questions posed, a record of useful online discussion posts, or any other substantial contribution. Simply attending class or doing well on assignments does not qualify.

Academic Integrity

Due to the updated evaluation scheme in this class, the only academic integrity concerns relate to cheating on quizzes and exams. However, all quizzes and exams are to be completed individually. This will be partially enforced by a ban on electronics during quizzes and exams, except for a standard calculator.

Consulting with ChatGPT and other AI assistants, as well as your classmates, teaching staff, or anyone else on assignments is strongly encouraged. You may freely copy material from online sources, hire tutors to help with your homework, and leverage artificial intelligence to produce your homework solutions, in any way you see fit.

Study Suggestions

This class teaches a combination of concepts and practice, including many practical skills that are not taught in OS textbooks. The best way to learn is to read and struggle through the xv6 kernel code, and to work through the assignments with the goal of understanding the part of the OS that each assignment touches.

While you are highly encouraged to use whatever means you want to learn the required concepts and skills, including any form of homework help you want, please be aware of the pitfalls of not doing the work yourself, in a class focused on learning by doing. Leaning on ChatGPT or a fellow student to complete your assignments is a recipe for poor quiz and exam performance.

The grading policy is explicitly designed to eliminate the pressure to simply finish an assignment by any means, and instead encourage you to try your best to understand the things the assignment is there to teach you.

Attendance and Missed Work

There is no attendance requirement, except to take in-class quizzes and exams. That said, attendance is highly encouraged, both to maximize what you get out of class yourself by participating in the class discussion, and to help make the class better for everyone by asking questions when there is something you don't fully understand. Lectures will be recorded.

The grading policy above is quite accommodating regarding missed work. Deadlines generally cannot be extended on a per-person basis as we will have a quiz immediately following, and solutions to the assignments will be discussed in detail after the deadline. Please make sure you can attend the midterm and final exams, the dates of which are posted above.

Classroom Norms

Our class (in person and online) will follow the CS Code of Conduct (https://www.cs.uic.edu/~grad/CS_Code_of_Conduct.pdf). If you are not adhering to our course norms, a case of behavior misconduct will be submitted to the Dean of Students and to the Director of Undergraduate Studies in the department of Computer Science. If you are not adhering to our course norms, you will not get full credit for your work in this class. For extreme cases of violating the course norms, credit for the course will not be given.

Inclusive Learning Environment

UIC values diversity and inclusion. Regardless of age, disability, ethnicity, race, gender, gender identity, sexual orientation, socioeconomic status, geographic background, religion, political ideology, language, or culture, we expect all members of this class to contribute to a respectful, welcoming, and inclusive environment for every other member of our class. If there are aspects of the instruction or design of this course that result in barriers to your inclusion, engagement, accurate assessment or achievement, please notify me as soon as possible.

Disability Services

UIC is committed to full inclusion and participation of people with disabilities in all aspects of university life. If you face or anticipate disability-related barriers while at UIC, please connect with the Disability Resource Center (DRC) at drc@uic.edu, via email at drc@uic.edu, or call (312) 413-2183 to create a plan for reasonable accommodations. In order to receive accommodations, you will need to disclose the disability to the DRC, complete an interactive registration process with the DRC, and provide me with a Letter of Accommodation (LOA). Upon receipt of a LOA, I will gladly work with you and the DRC to implement approved accommodations.

Topics Covered (tentative)

1. Boot and initialization
2. Abstractions and Interfaces (shell, kernel, isolation, system calls, interrupts, signals)
3. Memory (virtual memory, memory allocators, COW, dedup)
4. Processes (process creation, CPU scheduling, threads)
5. Concurrency (synchronization, locks, RCU)
6. File Systems (VFS, Persistent Storage, Consistency)

Related Courses

- MIT 6.828: <https://pdos.csail.mit.edu/6.828/>